

Ingeniería del Software II

Práctica #7 – Dataflow Analysis Interprocedural

Ejercicio 1

Sea el siguiente programa:

```
1  int multByTwo(int x) {
2    int result = 2*x;
3    return result;
4  }
5  void main() {
6    int x,y;
7    x = 5;
8    y = multByTwo(x);
9    x = 0;
10   y = multByTwo(x);
11   print(x, y);
12 }
```

- Construir el CFG de las funciones `multByTwo` y `main`.
- Sea el reticulado $flat(\{< 0, Z, > 0\})$, ejecutar el Zero Analysis sobre ambas funciones. ¿Cuál es valor abstracto para `x` y `y` que es calculado por el análisis a la entrada del nodo 11 (`print(x,y)`)?
- Realizar nuevamente el Zero Analysis pero considerando llamados y retornos. Comparar el valor abstracto para `x` y `y` a la entrada del nodo que contiene `print(x,y)`.
- Ejecutar el zero analysis luego de aplicar cloning a la función `multByTwo`.
- Ejecutar el zero analysis usando cadenas de llamadas con `k=1`.
- Ejecutar el zero analysis usando functional context.

Ejercicio 2

En el programa anterior reemplazar `multByTwo` por:

```
1  int multByTwo(int x) {
2    return multByTwo_2(x);
3  }
4  int multByTwo_2(int x) {
5    int result = 2*x;
6    return result;
7  }
```

¿Cuál es el efecto en los items d) e) y f) del ejercicio anterior?

Ejercicio 3

Para el siguiente programa:

```
1  int inc(int a) {
2    return a+1;
3  }
4  void main() {
5    int y = input();
```

```

6   int x = 0;
7   while(y>0) {
8       x = inc(x);
9       y = y-1;
10  }
11  print(x);
12  }

```

- Construir el CFG de `inc` y `main`.
- Calcular el análisis interprocedural usando el dominio del signo sin contextos.
- ¿Qué valor calcula el análisis para el `print`?
- Recalcular el dataflow luego de aplicar clonning a la función `inc`.
- Recalcular el dataflow usando cadenas de llamadas con $k=1$.
- Recalcular el dataflow usando functional context.

Ejercicio 4

Para el siguiente programa:

```

1  int g(int n) {
2      return n + 2;
3  }
4
5  void main() {
6      int a, b;
7      a = 5;           // a es Impar
8      b = g(a);
9      a = 4;           // a es Par
10     b = g(a);
11     print(b);
12 }

```

Se utilizará un **análisis de paridad** con el siguiente reticulado: **Par** (**P**), **Impar** (**I**), **Top** (**⊤**), **Bottom** (**⊥**) (notar que deben definir $x = \text{cte}$, y la suma)

- Construir el CFG para las funciones `g` y `main`.
- Calcular el análisis de dataflow interprocedural de paridad **sin contexto de llamada**. Mostrar los valores de *in* y *out* para cada punto del programa.
- ¿Qué valor tiene `out[10](b)`? Es decir, ¿cuál es la paridad de la variable `b` al final de la línea 10 según este análisis?
- Recalcular el dataflow utilizando **cadenas de llamadas con $k=1$** . Indicar el valor de `out[10](b)` según este nuevo análisis.
- Recalcular el dataflow utilizando **contexto funcional**. ¿Cuál es el valor final de `out[10](b)`?

Ejercicio 5

Analizar el siguiente programa usando el dominio $Var \mapsto Sign$:

```
1  int incrementar(int n) {  
2      return n + 3;  
3  }  
4  
5  int duplicar(int m) {  
6      return m * 2;  
7  }  
8  
9  void main() {  
10     int x;  
11     x = 7;  
12     y = incrementar(x);  
13     x = duplicar(0);  
14     y = incrementar(x);  
15     print(y);  
16 }
```

- a. Construir el CFG para todas las funciones
- b. Calcular el análisis interprocedural **sin contexto**. Mostrar valores de *in* y *out* en cada punto.
- c. ¿Qué signo tiene y en `print(y)`? ($\text{in}[15](y)$)
- d. Repetir b y c con **cadenas de llamadas k=1**.